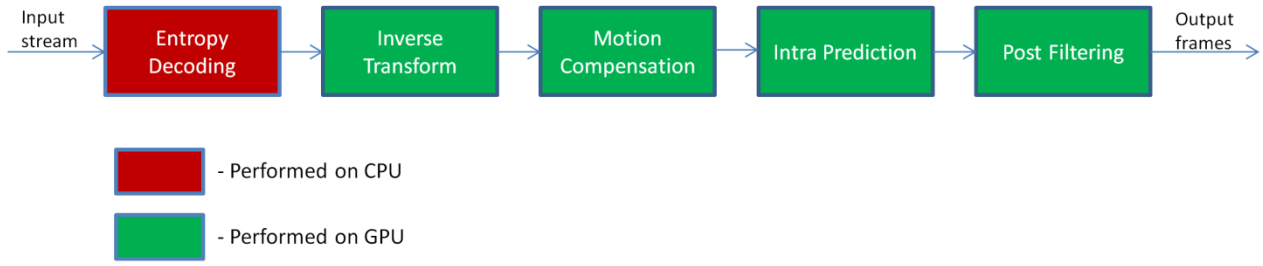
AV1 Software Decoder for Xbox One Platform

This project is an open source implementation of a GPU-accelerated AV1 decoder for the Microsoft Xbox One platform.

1. Implementation overview

AV1 decoder for Xbox is designed, developed and tested for Microsoft Xbox One game consoles. Evaluation of the compute resources available on those game consoles showed that to get the maximum performance level it is necessary to move as much as possible computations to GPU. This caused the following structure of the decoder implementation:



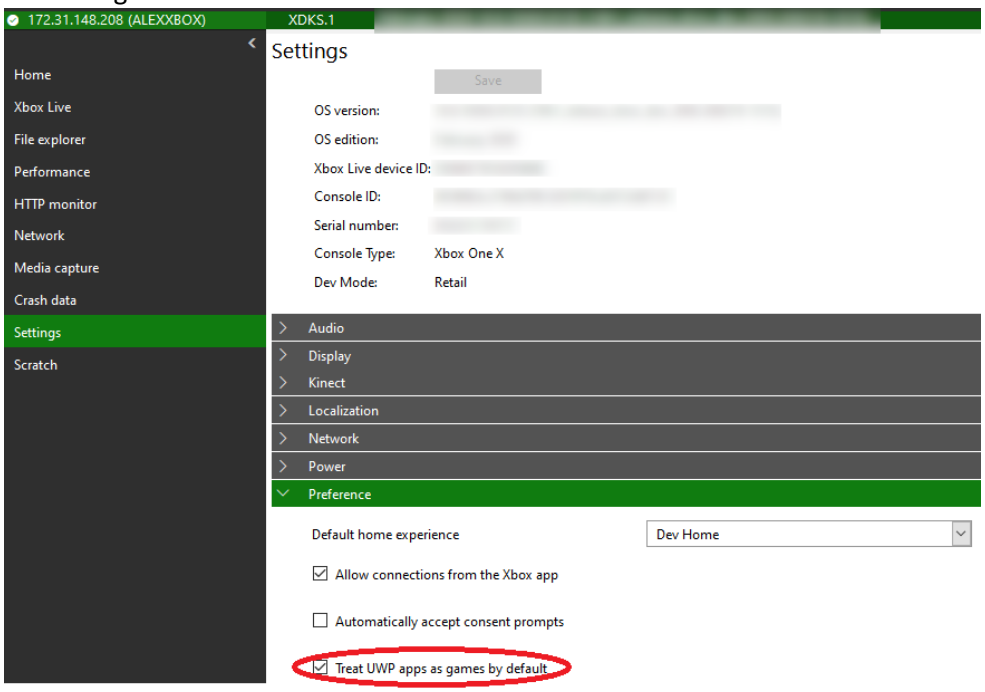
CPU and GPU processing are pipelined, so when GPU is working on frame N, CPU is performing entropy deoding for the next N+1 frame.

2. System requirements

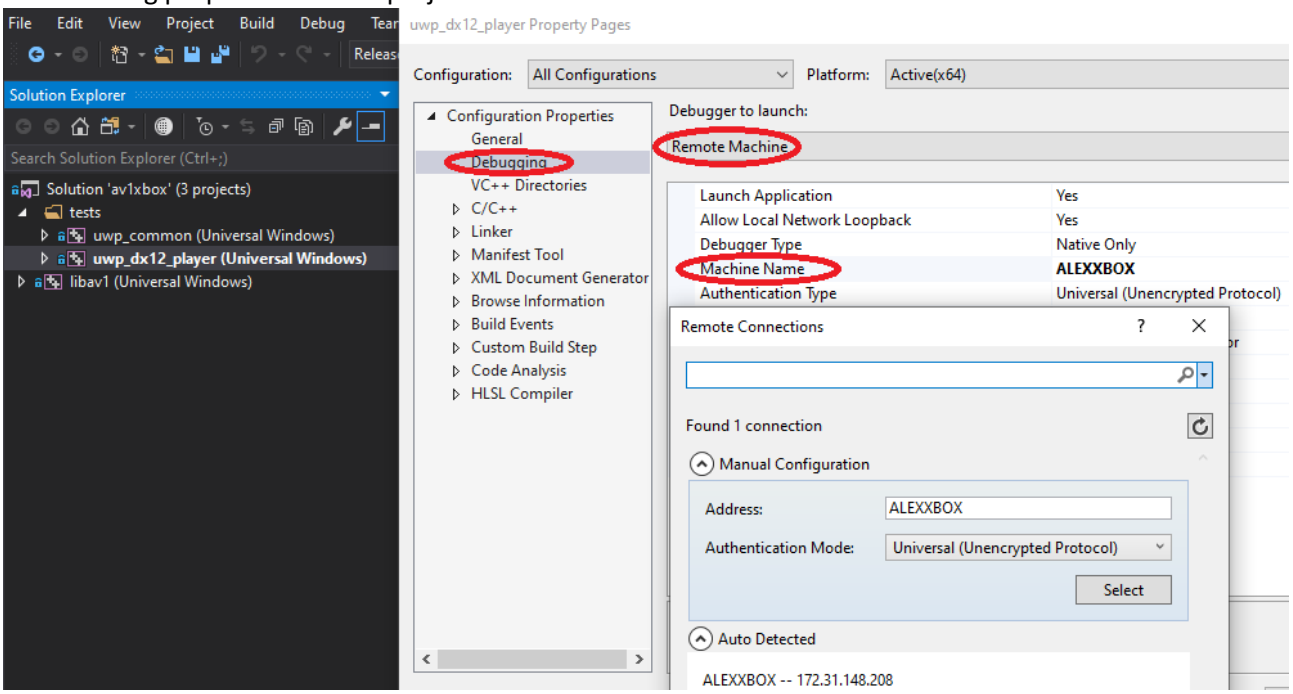
1. Hardware: Xbox One, Xbox One S, or Xbox One X
2. Host application to run the decoder library: Universal Windows Platform (UWP) application in **game** mode. Actually this mode is required to get access to DirectX 12 compute shaders.

3. Build instructions

1. Switch Xbox into developer mode
2. Turn on “game” mode on Xbox Device Portal:

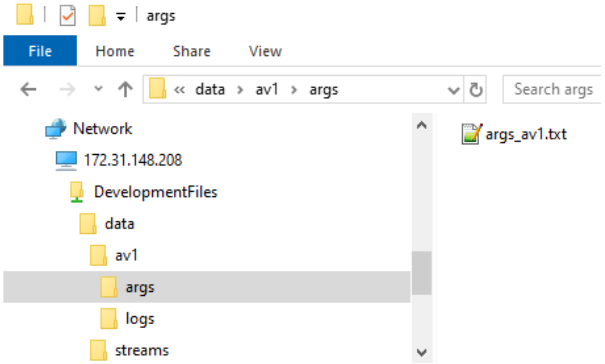


3. Open in Visual Studio the AV1 decoder solution, and set av1xboxplayer project as startup. Select the following properties for this project:



4. Execution of the test player

To configure the test player the file “args_av1.txt” with decoder’s command line strings is used. The file is stored on console in the following location (created manually as text file):



For the first time Xbox storage may be accessed using File Explorer from Xbox Device Portal, then Windows Explorer may be used as well.

As XB1 doesn’t have console output, all information about an execution is written into log files stored in the folder “..\DevelopmentFiles\data\av1\logs”

5. Command line options

Below are the sample command line strings (they may be stored in one args_av1.txt file):

#-dir \av1\streams -md5_frame_check -t 4 -frate -1
#-i "\av1\streams\stream_name.mp4" -t 4 -limit 15000
-i "\av1\streams\ stream_name.mp4" -t 4 -frate 60 -limit 200 -dropinfo
-dir \av1\streams\ -t 4 -try10x3 -frate -1

Command line options of the test player:

Command line parameter name	Command line parameter description
#	Comment

i	Input file path
o	Path to store decoded YUV file
dir	Path to the directory to decode
t	The number of decoding threads. Recommended t=6
loop	The number of times to decode the stream
limit	The number of output frames to decode
progress	Display decoding time for every frame
save_md5	Save md5 for each frame into a text file with the same name as input stream
md5_frame_check	Calculate md5 for each frame and compare it with the information from md5 file stored in the same folder with the same name as input stream
start	Start decoding from frame with the specified number
frate	Outputting frame rate. Required to verify frame drops at the specified frame rate. 0 - retrieved from the stream header (for webm only), -1 - maximum frate,
dropinfo	Outputs information about all dropped frames at specified frate
queue_size	The size of the internal player frames queue. Default 16. Required to reduce drops on heavy streams (depends on Xbox model/performance).
conformance	Calculate md5 for the whole stream and compare it with the information from md5 file stored in the same folder with the same name as input stream
try10x3	Use 10_10_10_2 display mode. Useful to reduce memory usage in HDR mode. Default false
r	Recreate decoder for every file

Project Contributions

This project is not accepting contributions at this time, but we encourage any interested parties to fork the project in order to make any desired modifications.